

Università
degli Studi
di Padova

University of Padua

Department of Mathematics “Tullio Levi-Civita”

Formal Methods for Cyber-Physical Systems

ASSIGNMENT 1

Luca Monetti

Part 1

Assignment

The primary objective of the first part of the assignment is the implementation of a symbolic algorithm for invariant verification using BDDs (Binary Decision Diagrams) to interact with regions. Given a property, denoted as `spec`, the algorithm determines whether it holds for all reachable states of the loaded model.

If a specification is not an invariant of the model a counter-example must be returned. A counter-example is a `tuple` of alternating states and input that represent an actual execution of the system that start from an initial state and lead to a state that violates the specification.

The output must be a tuple of 2 elements:
- Result: a `boolean` value that is `True` if `spec` is an invariant, `False` otherwise.
- Counter Example: `None` if `spec` is an invariant, a counter-example otherwise.

Logic

An invariant of the model is a property φ that holds for every reachable state. The verification process is built on this principle: to prove that φ is an invariant of the system the algorithm must demonstrate that $\neg\varphi$ is unreachable from any initial state.

That leads to two possible outcome:
- Invariant Holds: if the algorithm explores all the reachable states without finding one in which $\neg\varphi$ is `True` then φ is an invariant of the model.
- Violation: if the algorithm find a reachable state where $\neg\varphi$ holds then the property φ is not an invariant.

Implementation

The verification process begins with the conversion of the specification `spec` into a BDD using the provided function `spec_to_bdd(model, spec)`. This allows for the application of symbolic algorithm for invariant verification.

The algorithm proceeds through the following phases:
- The sets `reach` and `new` are initialized with the model's initial states.
- At each iteration, the set `new` is evaluated. If `new` is empty or violates the specification φ then the exploration is complete.
- If no violations are found and `new` is not empty the algorithm computes the post-image of `new` removing all the already checked states. The newly discovered states are included in `reach`.
- After the last iteration of the loop there are 2 cases:
  - If there is an invalid state inside `new` then the algorithm returns `False` and computes a counter-example.
  - If `new` is empty the specification φ holds for every reachable state; therefore, it is an invariant.

```
1  def check_explain_inv_spec(spec):
2      # 1. Setup the bdds
3      model = pynusmv.glob.prop_database().master.bddFsm  # Model fsm
4
5      spec_bdd = spec_to_bdd(model, spec)                 # Bdd of spec
```

```
    negated_spec_bdd = -spec_bdd                        # Bdd of ~spec

    # 2. Explore the regions
    reach = model.init                                  # Already explored states
    new = model.init                                    # States to explore

    trace = [reach]                                     # Trace used to save the execution

    while new.isnot_false() and not new.intersected(negated_spec_bdd):

        new = model.post(new) - reach                   # Remove already checked states
        reach = reach + new                             # Add new states to reach

        trace.append(new)                               # Add reached region to trace

    if (not new.intersected(negated_spec_bdd)):
        # The formula is satisfied on every reachable state
        return (True, None)

    # 3. Build the counter-example
    invalid_states = new * negated_spec_bdd             # States in new that violate the spec
    target = model.pick_one_state(invalid_states)       # State that violate the spec

    counter_example = compute_counter_example(model, target, trace)

    # The property is not an invariant, return a counter-example
    return (False, tuple(counter_example))
```

Counter example generation

When a specification is violated, a counter-example must be provided to demonstrate the failure.

The algorithm maintains a `trace` list where each element represents the region reached on that iteration starting from the previous one. The execution is then reconstructed via backwards traversal.

The function `compute_counter_example(model, trace, spec)` implements this operation throught the following steps:

- The algorithm pick a state `current` from the set of invalid states, computed by the intersection `trace[-1] * spec`, and insert it into the `counter_example` list.
- The `trace` is then traversed in reverse starting from the second to last element. At each iteration one state is selected from the current region and added, with the curresponding input, to `counter_example`.

- The procedure returns a valid execution of the model that leads to an invalid state.

```python
def compute_counter_example(model, trace, neg_spec):
    invalid_states = trace[-1] * neg_spec                          # States that violate spec
    current = model.pick_one_state(invalid_states)              # Current considered value

    pre_states = model.pre(current)   # Compute states that can reach current in one step
    counter_example = [current.get_str_values()]

    # Check if the model uses inputs
    has_inputs = not not len(model.bddEnc.inputsVars)

    # Interate over the trace starting from the second last
    for current_set in reversed(trace[:-1]):

        # Get a state that has a transition to current
        prev_state = model.pick_one_state(current_set * pre_states)

        # Get the input that caused the transition
        if has_inputs:
            trans_inputs = model.get_inputs_between_states(prev_state, current)
            input_values = model.pick_one_inputs(trans_inputs).get_str_values()
            counter_example.insert(0, input_values)
        else:
            counter_example.insert(0, {})

        # Add the picked state
        counter_example.insert(0, prev_state.get_str_values())

        # Update invariants
        current = prev_state
        pre_states = model.pre(current)

    return counter_example
```

Correctness

This section provides a formal demonstration of correctness for the implemented algorithm. The algorithm is composed by two main loops:

- Reachable state exploration
- Counter-example generation

The correctness of each phase is demonstrated using the Loop invariant technique.

Exploration of the reachable states

The exploration phase identifies all the reachable states while checking for property violations.

Loop invariants

Three invariants are maintained throughout the execution in the first loop:

- `reach`: representing the region of all the already reached states without duplicates.
- `new`: represent the region of all the newly found reachable states.
- `trace`: an ordered list where each element represents a regions of all the states reached at each iteration.

Base case

Initially `reach`,`new` and `trace` are initialized with the initial states of the model, therefore all invariants hold.

Inductive step

The loop condition checks if `new` is not empty, in that case there are still new states to explore and the loop body is executed.

For each iteration i:

- `new` is updated by computing the post-image of the previous `new` and subtracting all the states that are already present in `reach`. Therefore `new` contains only states reachable from the $i-1$ iteration region that are not already in `reach`.
- `reach` is updated to include the current set `new`, therefore it contains all the reachable states found so far.
- `trace` appends the current set `new`, therefore is a list of $i+1$ elements representing all the regions reachable from the previous region.

Termination

The loop invariants hold for the base case and for the inductive case. Therefore by induction the loop invariants hold for each iteration of the loop.

Since the state space of the model is finite the algorithm will eventually explore all the reachable states. Therefore there are only two possible execution:

1. `new` becomes empty, all the reachable states have been explored and none of them violate the specification, so the algorithm returns `(True, None)`.
2. `new` intersected with `neg_spec_bdd` is not empty, there exists at least one newly reached state that violates the specification φ, so the algorithm returns `False` along with a counter-example.

Counter-example generation

If a violation is detected, a second loop reconstructs a valid execution path from one initial state to the violating state.

Loop invariant

Two invariants are maintained throughout the execution in the second loop:

- `counter_example`: a list of alternating states and inputs representing a valid execution of the system that lead to a invalid state.
- `current`: the current state being added to the counter-example list.

Base case

Initially `current` is initialized with a state picked from the set of invalid states, therefore it is a state that violates the specification. The `counter_example` list is initialized with this state. Therefore the invariants hold.

Inductive step

The algorithm iterate backwards through the `trace` list, starting from the second to last element. At each step i:

- A predecessor state is selected from a list of states that are in `trace[i]` and can reach `current` in one transition.
- `counter_example` is updated by prepending this predecessor and the corresponding input.
- `current` is updated to this predecessor.

Termination

The invariants hold for the base case and for the inductive case. Therefore by induction the loop invariants hold for each iteration of the loop.

The trace list is finite, since the model is finite and the algorithm explores only reachable states. Therefore the loop will terminate after a finite number of steps. There are only one possible execution: the algorithm returns the `counter_example` list.

Part 2

Assignment

The second part of the assignment requires the implementation of a symbolic algorithm for the verification of a specific class of LTL formulas called *response formulas*. These formulas follow the structure:

$$\square(f \rightarrow \diamondsuit g)$$

In this context, both f and g are Boolean combinations of base formulas with no temporal operators.

In the following pages will be described an implementation of the function `check_examplain_response_spec(spec)` that determines id the provided `spec` is satisfied by the loaded model.

If a specification is not satisfied by the model a counter-example must be returned. A counter-example is a `tuple` of alternating states and input that represent an actual execution of the system that start from an initial state and lead to a state that violates the specification.

The output must be a tuple of 2 elements:
- Result: a `boolean` value that is `True` if `spec` is satisfied, `False` otherwise.
- Counter Example: `None` if `spec` is satisfied, a counter-example otherwise.

Logic

The algorithm addresses *response formula*, which are defined by the requirement:

Every time that f occours, g must eventually happen at the same time or afterwards.

As a liveness property, a response formula is evaluated over infinite computations. To prove that the formula is falsified a counter-example that satisfies the negation of the property must be provided. A path in witch f holds but g never happens again.

In a finite state system such path is composed by two section:
1. **Prefix**: A sequence of states leading to a state where f is satisfied.
2. **Loop**: An infinite cycle originating from the prefix, within which g is always `False`.

Implementation

The verification process begins by validating the structure of the specification using the provided function `parse(spec)`. If the function returns None, the specification is not a valid response formula, and the procedure terminates.

Once the structure is validated, the formulas f and $\neg g$ are converted into BDDs using the provided function `spec_to_bdd(model, spec)`.

Iterative cycle detection

The algorithm will operate only on the reachable states of the model, identified by `model.reachable_states`. The variable `recur` is initialized within this set, specifically to represent all reachable states where f holds and g is `False`.

The algorithm then preceeds to identify a cycle where g remains `False`:
- At each iteration `recur` is evaluated. If `recur` is empty then a cycle is not found and the algorithm returns `(True, None)`.

- Otherwise the algorithm compute `pre_reach`, which consists of all states capable of reaching the current `recur` in one or more transition. The set `new` is computed to identify states that remains part of a potential cycle.
- If a cycle is found, the algorithm build a counter-example using the function `compute_counter_example(model, recur, pre_reach)`, which will be describe in the next section.

```
1  def check_explain_response_spec(spec):
2    # 1. Check the formula structure
3    react = parse(spec)
4    if (react == None):
5        return None
6
7    # 2. Setup the bdds
8    model = pynusmv.glob.prop_database().master.bddFsm
9
10   f_spec, g_spec = react
11
12   f_bdd = spec_to_bdd(model, f_spec)                    # BDD for formula f
13   neg_g_bdd = spec_to_bdd(model, ~g_spec)               # BDD for formula ~g
14
15   # 3. Cycle detection
16   reach = model.reachable_states                        # Reachable states
17   recur = reach * neg_g_bdd * f_bdd                     # Possible starting point for a cycle
18
19   while recur.isnot_false():
20       pre_reach = pynusmv.dd.BDD.false()                # States that can reach recur
21       new = model.pre(recur) * neg_g_bdd                # Newly discovered states
22
23       while new.isnot_false():
24
25           pre_reach = pre_reach + new
26
27           if recur <= pre_reach:
28               # A cycle is found
29               return (False, compute_counter_example(model, recur, pre_reach))
30
31           new = (model.pre(new) - pre_reach) * neg_g_bdd  # Added only state where ~g is True
32
33       recur = recur * pre_reach
```

```
34
35    # The formula is satisfied by the model
36    return (True, None)
```

Counter example generation

A counter-example is an execution of the system that leads to a state where the formula is not satisfied. It is composed of 2 parts:

1. **Prefix**: A sequence of states leading to a state where f hold.
2. **Loop**: An infinite cycle in which g is always `False`.

Those two parts are computed by `compute_prefix` and `compute_loop` and then combined.

In the end the algorithm return an execution that start from an initial stata, reach a state where f holds and g is `False` and then proced with a cycle whithin which g is always `False`.

```
1  def compute_counter_example(model, recur, pre_reach):
2    # Analyze each state of recur
3    for s in model.pick_all_states(recur):
4
5        r = pynusmv.dd.BDD.false()              # Reached states
6        new = model.post(s) * pre_reach         # Newly valid state in post image
7
8        frontiers = [new]
9
10       # Compute frontiers
11       while new.isnot_false():
12           r = r + new
13           new = (model.post(new) * pre_reach) - r
14
15           frontiers.append(new)
16
17       r = r * recur
18
19       if s <= r:
20           prefix = compute_prefix(model, s)
21           loop = compute_loop(model, s, frontiers)
22           return tuple(prefix + loop)
```

Build the prefix

The prefix is reconstructed by identifying a path form the initial state set to the specific set identified as the entry point of the cycle. This procedure follows the previously discussed symbolic reachability approach.

Starting from model.init, the algorithm explores reachable states layer by layer. Each layer (or "frontier") is stored in a list until a frontier is reached that contains the designated starting state of the cycle.

Once the target state is located the sequence is constructed by traversing the frontiers in reverse. The resulting sequence is then assembled into the final `tuple`.

```
def compute_prefix(model, s):
  # Check if the model uses inputs
  has_inputs = not not len(model.bddEnc.inputsVars)

  reach = model.init                  # Already explored states
  new = model.init                    # States to explore

  frontiers = [reach]

  while (reach * s).is_false():       # Check if s is contained in reach

      new = model.post(new) - reach   # Find reachable states and remove already checked ones
      reach = reach + new             # Add new states to reach

      frontiers.append(new)           # Add reached region to frontiers

  # Build the path
  current = s

  pre_states = model.pre(current)
  path = [current.get_str_values()]

  # Interate over the frontiers starting from the second last
  for frontier in reversed(frontiers[:-1]):

      # Get a state that has a transition to current
      prev_state = model.pick_one_state(frontier * pre_states)

      # Get the input that caused the transition
      if has_inputs:
          trans_inputs = model.get_inputs_between_states(pre_states, current)
          inpute_values = model.pick_one_inputs(inputs).get_str_values()
          path.insert(0, input_values)
      else:
```

```
35            path.insert(0, {})
36
37        # Add the picked state
38        path.insert(0, prev_state.get_str_values())
39
40        # Update invariants
41        current = prev_state                # Update the current state
42        pre_states = model.pre(current)     # States that can reach current in one step
43
44    return path
```

Build the loop

The reconstruction of the infinite cycle is a more complex precedure that requires to identify a sequence that begins and ends at the same state.

The algorithm initially identify the minimum number of steps required to reach the state `s` withing the candidate region.

The algorithm mantains two invariant:

- `path`: a sequence of alternating states and inputs.
- `current`: the state currently being precessed.

At each iteration the algorithm select a predecessor state from the preceding frontier that can reach `current` in a single transition. The state is prepend to the output with its corresponding input.

```
1  def compute_loop(model, s, frontiers):
2    # Check if the model uses inputs
3    has_inputs = not not len(model.bddEnc.inputsVars)
4
5    # Find the minimum value of k
6    for k in range(len(frontiers)):
7        if s <= frontiers[k]:
8            break
9
10   # Reconstruct the execution
11   path = [ s.get_str_values() ]                    # Cycle execution
12   current = s                                      # Current Value
13
14   for i in range(k - 1, -1, -1):
15
16       # Get a state that has a transition to current
17       prev_states = model.pre(current) * frontiers[i]
```

```
        prev_state = model.pick_one_state(prev_states)

          # Get the input that caused the transition
        if has_inputs:
            trans_inputs = model.get_inputs_between_states(pre_states, current)
            inpute_values = model.pick_one_inputs(inputs).get_str_values()
            path.insert(0, input_values)
        else:
            path.insert(0, {})

        # Add the picked state
        path.insert(0, prev_state.get_str_values())

        # Update invariants
        current = prev_state                # Update the current state
        pre_states = model.pre(current)     # States that can reach current in one step

    # Get inputs that starts the loop
    if has_inputs:
        inputs = model.get_inputs_between_states(s, current)
        path.insert(0, model.pick_one_inputs(inputs).get_str_values())
    else:
        path.insert(0, {})

    # Add s at the beginning of the loop
    path.insert(0, s.get_str_values())

    return path
```

Correctness

This section provides a formal demonstration of correctness for the implemented algorithm. The algorithm is composed by three main loop that can be repeated in different functions:

- Cycle identification
- Breadth-First Search
- Counter-example generation

The correctness of Breadth-First Search and Counter Example generation are discussed in the previous part. The correctness of the cycle identification can be proved using the Loop invariant technique.

Cycle identification

The cycle identification phase identifies a set of states thath violates the liveness property $G(f \rightarrow Fg)$ by searching for a cycle where f has occourred but g never follows.

Loop invariants

The cycle detection is done by an outer loop that reduce the set of potential states that can start a cycle and an inner state that computes the set of states that can reach each of those states.

Two main invariant are maintained in the inner loop:

- `pre_reach`: represents the region of all states that can both reach the current `recur` set in one or more transition and satisfies $\neg g$.
- `new`: represent the region of all the newly found states that can both reach the current `recur` in one or more transition and satisfy $\neg g$.

Base case

Initiallt `pre_reach` is empty and `new` is initialized to `model.pre(recur) * neg_g_bdd`. The invariants hold for the base case.

Inductive step

The loop condition checks if `new` is not empty. In that case there are still states that can starts a cycle where $\neg g$ holds.

For each iteration i:

- `pre_reach` is updated to include `new`. It now contains all the states that can reach the current `recur` in one or more transition.
- `new` is updated with the pre-image of the current `new` by subtracting states already in `pre_reach` and restrinting its value to those that respect `not_g_bdd` ($\neg g$). It now contains all the states that can reach the current `recur` in $i + 1$ steps.
- If `recur` becoms a subset of `pre_reach` there is a cycle.

Termination

The loop invariants hold for the base case and for the inductive case. Therefore by induction the loop invariants hold for each iteration of the loop.

Since the state space of the model is finite the set of states `pre_reach` cannot grow indefinitely.

Therefore there are only two possible execution:

1. `new` becomes empty. This means that the current `recur` cannot reach itself. The outer loop will reduce `recur` and tries again. If `recur` becomes empty the algorithm returns `(True, None)`.
2. If the condition `recur <= pre_reach` becomes true a cycle exists. The algorithm returns `False` along with a counter-example.